# 14. STARK 프로토콜 이론 설명 (STARK Protocol Theory)

## 개요

STARK(Scalable Transparent ARgument of Knowledge)는 SP1의 핵심을 이루는 영지식 증명 시스템입니다. 이 문서는 STARK 프로토콜의 수학적 기초, 보안 가정, 그리고 SP1에서의 구체적인 구현 방식을 상세히 설명합니다.

## STARK의 수학적 기초

### 1. 기본 개념

STARK는 다음과 같은 수학적 구조를 기반으로 합니다:

- **유한체(Finite Field)**: 모든 연산이 유한체 위에서 수행됩니다
- **다항식(Polynomials)**: 계산 추적을 다항식으로 인코딩합니다
- **Reed-Solomon 코드**: 오류 정정 코드를 통한 근사 증명을 제공합니다
- **Merkle 트리**: 다항식 커밋을 위한 암호학적 해시 구조입니다

### 2. AIR (Algebraic Intermediate Representation)

SP1에서 STARK는 AIR 시스템을 통해 구현됩니다:

```
// SP1의 AIR 구조 예시
pub trait Air<AB: AirBuilder> {
fn eval(&self, builder: &mut AB);
```

```
}

// 제약 조건 정의
impl<AB: AirBuilder> Air<AB> for CpuChip {
fn  eval(&self, builder: &mut AB) {
// 산술 제약 조건
builder.assert_eq(a + b, c);
// 메모리 일관성 제약 조건
builder.when(is_memory_op).assert_eq(
memory_value,
expected_value
);
}
}
```

### 3. 실행 추적 (Execution Trace)

실행 추적은 프로그램 실행의 각 단계를 행렬로 표현합니다:

```
T = [
[pc₀, reg₀, mem₀, ...], // 첫 번째 사이클
[pc₁ , reg₁ , mem₁ , ...], // 두 번째 사이클
...
[pcₙ, regₙ, memₙ, ...] // 마지막 사이클
```

]

## 다항식 커밋 스킴 (Polynomial Commitment Scheme)

### 1. FRI 프로토콜 (Fast Reed-Solomon Interactive Oracle Proof)

SP1은 FRI 프로토콜을 사용하여 다항식 커밋을 구현합니다:

**FRI 폴딩 과정**

```
// FRI 폴딩 이벤트 구조
pub struct FriFoldEvent<F> {
pub base_single: FriFoldBaseIo<F>,
pub ext_single: FriFoldExtSingleIo<Block<F>>,
pub ext_vec: FriFoldExtVecIo<Block<F>>,
}

// 폴딩 연산: f(x) → f'(x²)
// f'(x) = (f(x) + f(-x))/2 + α·(f(x) - f(-x))/(2x)
```

**FRI 검증 과정**

1. **커밋 단계**: 다항식을 Merkle 트리로 커밋
2. **쿼리 단계**: 무작위 점에서 다항식 값 요청
3. **일관성 검증**: 폴딩 관계 확인

### 2. 배치 FRI (Batch FRI)

여러 다항식을 동시에 처리하여 효율성을 향상시킵니다:

```
pub struct BatchFRIEvent<F> {
pub base_vec: BatchFRIBaseVecIo<F>,
pub ext_single: BatchFRIExtSingleIo<Block<F>>,
pub ext_vec: BatchFRIExtVecIo<Block<F>>,
}

// 배치 처리: Σᵢ αⁱ·fᵢ(x) 형태로 결합
```

## 영지식성과 건전성 증명

### 1. 영지식성 (Zero-Knowledge)

STARK의 영지식성은 다음과 같이 보장됩니다:

#### 무작위화 (Randomization)

```
// 증명자가 추가하는 무작위 다항식
let randomizer_poly = generate_random_polynomial(degree);
let blinded_trace = original_trace + randomizer_poly;
```

#### 시뮬레이터 존재성

- 비밀 정보 없이도 구별 불가능한 증명을 생성할 수 있는 시뮬레이터가 존재합니다

- 이는 증명이 비밀 정보를 누출하지 않음을 보장합니다

### 2. 건전성 (Soundness)

건전성은 거짓 명제에 대한 증명이 높은 확률로 거부됨을 의미합니다:

**근사 정도 (Proximity)**

```
// 코드워드와의 거리 측정
fn proximity_test<F: Field>(
codeword: &[F],
received: &[F],
rate: f64
) -> bool {
let distance = hamming_distance(codeword, received);
distance < (1.0 - rate) * codeword.len() / 2.0
}
```

**오류 확률**

- 각 FRI 라운드에서 오류 확률이 지수적으로 감소합니다
- 전체 오류 확률: `2^(-λ)` (λ는 보안 매개변수)

## SP1에서의 STARK 구현

### 1. 증명 생성 과정

```rust
impl<SC, A> MachineProver<SC, A> for CpuProver<SC, A> {
fn prove(&self, pk: &StarkProvingKey<SC>, records: Vec<A::Record>)
-> Result<MachineProof<SC>, Self::Error> {
// 1. 실행 추적 생성
let traces = self.generate_traces(&record);
// 2. 메인 커밋
let main_data = self.commit(&record, traces);
// 3. 순열 증명 생성
let permutation_challenges = challenger.sample_challenges();
let perm_traces = generate_permutation_traces(&traces, &challenges);
// 4. 몫 다항식 계산
let quotient_values = compute_quotient_values(&traces, &constraints);
// 5. 오프닝 증명 생성
let opening_proof = pcs.open(&traces, &quotient_values, &challenger);
Ok(MachineProof { shard_proofs })
}
}
```

## 2. 제약 조건 시스템

SP1은 다양한 유형의 제약 조건을 지원합니다:

**산술 제약 조건**

```
// 기본 산술 연산 제약
builder.assert_eq(a * b, c); // 곱셈
builder.assert_eq(a + b, c); // 덧셈

// 조건부 제약
builder.when(condition).assert_eq(a, b);
```

### 메모리 제약 조건

```
// 메모리 일관성 검증
builder.assert_memory_access(
address,
value,
is_read,
timestamp
);
```

### 순열 제약 조건

```
// 글로벌 상태 일관성
builder.assert_permutation(
local_values,
global_cumulative_sum
```

```
);
```

### 3. 최적화 기법

#### 샤드 분할 (Sharding)

```
// 큰 프로그램을 작은 샤드로 분할
let shards = split_execution_into_shards(execution_trace, shard_size);
let shard_proofs = shards.par_iter()
.map(|shard| prove_shard(shard))
.collect();
```

#### 병렬 처리

```
// 병렬 제약 조건 평가
let quotient_values = trace_domains
.into_par_iter()
.enumerate()
.map(|(i, quotient_domain)| {
compute_quotient_for_chip(chips[i], quotient_domain)
})
.collect();
```

## 보안 분석

## 1. 보안 가정

STARK의 보안은 다음 가정들에 기반합니다:

- **해시 함수의 충돌 저항성**: Merkle 트리 보안
- **이산 로그 문제**: 타원 곡선 기반 연산
- **Reed-Solomon 코드의 거리 특성**: 근사 증명 보안

## 2. 공격 모델

### 적응적 공격자 (Adaptive Adversary)

```
// 공격자가 이전 쿼리 결과를 바탕으로 다음 쿼리를 선택
fn adaptive_attack_resistance() {
// Fiat-Shamir 변환을 통한 비상호작용 증명
let challenge = hash(transcript);
// 공격자가 challenge를 예측할 수 없음을 보장
}
```

### 양자 공격 저항성

- STARK는 양자 컴퓨터에 대해서도 보안을 유지합니다
- 해시 기반 구조로 인한 양자 저항성

## 3. 구체적 보안 수준

```
// SP1의 보안 매개변수
const SECURITY_BITS: usize = 100; // 2^100 보안 수준
```

```rust
const FRI_QUERIES: usize = 84; // FRI 쿼리 수

const BLOWUP_FACTOR: usize = 8; // 코드 확장 비율
```

## 성능 특성

### 1. 복잡도 분석

- **증명자 시간**: O(n log n) (n은 실행 단계 수)
- **검증자 시간**: $O(\log^2 n)$
- **증명 크기**: $O(\log^2 n)$
- **설정 없음**: 신뢰할 수 있는 설정이 불필요

### 2. 실제 성능 측정

```rust
// 성능 벤치마크 예시

fn benchmark_stark_proof() {

let program_size = 1_000_000; // 100만 사이클

let start = Instant::now();

let proof = prover.prove(&program);

let prove_time = start.elapsed();

let start = Instant::now();

let is_valid = verifier.verify(&proof);

let verify_time = start.elapsed();

println!("증명 생성 시간: {:?}", prove_time);

println!("검증 시간: {:?}", verify_time);
```

```
    println!("증명 크기: {} bytes", proof.size());

}
```

## 결론

STARK 프로토콜은 SP1의 핵심 기술로서 다음과 같은 특징을 제공합니다:

1. **확장성**: 대규모 계산에 대한 효율적인 증명
2. **투명성**: 신뢰할 수 있는 설정 불필요
3. **양자 저항성**: 미래의 양자 컴퓨터에 대한 보안
4. **실용성**: 실제 애플리케이션에서 사용 가능한 성능

이러한 특성들이 결합되어 SP1이 범용적이고 실용적인 영지식 가상 머신을 구현할 수 있게 합니다.